Online Attendance System

Charlie Le Geyt | K1209664

Project Supervisor: Graeme Jones

Plagiarism Declaration

The following declaration should be signed and dated and inserted directly after the title page of your report:

Declaration

I have read and understood the University regulations on plagiarism and I understand the meaning of the word *plagiarism*. I declare that this report is entirely my own work. Any other sources are duly acknowledged and referenced according to the requirements of the School of Computer Science and Mathematics. All verbatim citations are indicated by double quotation marks ("…"). Neither in part nor in its entirety have I made use of another student's work and pretended that it is my own. I have not asked anybody to contribute to this project in the form of code, text or drawings. I did not allow and will not allow anyone to copy my work with the intention of presenting it as their own work.

Date 23/04/17

Signature _____

ABSTRACT 4

1 INTRODUCTION 4

1.1 THE PROBLEM 4
1.2 AIMS AND OBJECTIVES 4
1.3 SOLUTION 4

2 LITERATURE REVIEW 5

2.1 INTRODUCTION 5
2.2 WHAT ARE DESIGN PRINCIPLES AND PATTERNS? 5
2.3 DESIGN PATTERNS AND PRINCIPLES IN PROGRAMMING LANGUAGES 5
2.4 DESIGN PATTERNS AND PRINCIPLES AND THE WEB 6
2.5 CONCLUSION 7

3 PROJECT METHODOLOGY 9

3.1 PROJECT METHODOLOGIES 9
3.2 CHOSEN METHODOLOGY 9

4 USER REQUIREMENTS ANALYSIS 10

4.1 SWOT ANALYSIS 10
4.2 CONCLUSION 11
4.3 STAKEHOLDER ANALYSIS 11
4.4 REQUIREMENTS ANALYSIS 13

5 DESIGN 19

5.1 DATABASE DESIGN 19
5.2 WEB INTERFACE DESIGN 27
5.3 CONCLUSION 28
5.4 CONCLUSIONS 36

6 IMPLEMENTATION 37

6.1 TECHNOLOGIES AND LANGUAGES 37
6.2 DATABASE CREATION 40
6.3 INTERFACE CREATION 40
6.4 ISSUES AND PROBLEMS ENCOUNTERED 43

6.5 CONCLUSION 43

7 TESTING 44

7.1 TESTING STRATEGY 44
7.2 TESTING METHOD 44
7.3 TESTING 45

8 REVIEW 47

8.1 PROJECT EVALUATION 47
8.2 PERSONAL EVALUATION 47
8.3 FUTURE DEVELOPMENTS 48
8.4 CONCLUSION 48

9 REFERENCES 49

10 APPENDIX 50

Abstract

The report follows the system life cycle of the Online Student Attendance System project. The system life cycle involves analysis and planning, design, implementation, testing and deployment. The report will explore and explain each stage of the cycle and include any justifications that are needed.

The aim of the project is to design and build an online attendance system for managing student attendance to lectures and workshops. Part of the aim is to design an alternative system to the current paper based system currently used. The system will contain a back-end database connected to a front-end web site.

1 Introduction

1.1 The Problem

The main problem this project is trying to solve is improving on the current paper based system. Currently attendance at lectures and workshops is taken by the lecturer printing off and handing around a paper list with each student timetabled for the session. This is both time consuming and slow to complete. The paper lists are collected by the lecturer then entered manually into an excel spreadsheet. This is time consuming for the lecturer and makes the sharing of the data difficult. Another issue is each lecturer will store the data differently and have different formatting.

1.2 Aims and Objectives

The overall aim is to improve the registration of students at lectures and workshops. This will be achieved through a series of objectives. Firstly, to solve the speed of registration, the system will be an online system. This will mean that multiple students will be able to be registered at the same time and stop students having to read through a long list of names to find their own name. Secondly, it will provide an effective way of storing each students' attendance without each lecturer having to manually enter the data into a spreadsheet. Thirdly, the online system will provide a platform for the collecting and sharing of the attendance data between hierarchy of the organisation.

1.3 Solution

The proposed solution is to design and create an online system for taking and monitoring attendance of students at lectures and workshops. The solution will be composed of a database and a web site. The system will allow users to log in using a username and password, then register their attendance at a timetabled session. The attendance for each user at each session will be stored in the database. Different user types will be able to view the data and, if permitted, generate reports from the data.

2 Literature Review

2.1 Introduction

There are many different forms of design principles and patterns, covering a range of topics ranging from architecture to programming languages. Design principles and patterns, whilst they differ from topic to topic, their role is mainly the same, to guide the design and implementation of an idea to make it the best it can be. This chapter will look at the design principles and patterns from a range of topics, but will primarily focus on the principles and patterns surrounding web development. Design patterns and principles have been designed to be reusable; they can be used in any system with similar functionality requirements [1].

2.2 What are Design Principles and Patterns?

Design principles and patterns have been around since they were conceived in the 1970's by an architect called Christopher Alexander. Alexander said "Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice" [7]. These concepts were designed for use in an architectural setting, presenting 253 patterns of "user friendly" solutions to recurring problems [2]. In creating these concepts, Alexander had an overall goal to allow the inhabitants of the buildings to design the environments. This is the idea that evolved into the ideology of user-centred development within the software development lifecycle. One of the first programming languages to adopt this approach and become object-orientated was a set of languages known as Smalltalk-xx, where the xx are the last digits of the year in which it was finalized [3]. It was not until 1987 at the OOPSLA conference on object-orientation [4] that design patterns and principles were shown to be useful in software development. In the conference a group of experts used the Smalltalk language to create their own user interface.

The design principles and patterns used in the Smalltalk language have been used to influence more modern languages like Java, PHP, Python, Ruby, and others. However, with these more modern languages, more modern patterns and principles have been developed. This has led to different sets of design patterns and principles being applied to the same language, for example web design, Java development and others. The sets of patterns and principles can be applied to two areas of design: front-end design and back-end design. This means that one set is used on what the end user will see and interact with and the other is used on the code that is used to create the system.

2.3 Design Patterns and Principles in programming languages

There are many layers to the design patterns and principles relating to programming languages. These layers go from an overall pattern like MVC to a more focused view like the patterns described below.

Within the more focused patterns, there are 23 design patterns [8] that apply to programming languages. Each of these patterns falls into one of three categories: Creational, Structural or Behavioural [8], with each category representing an aspect that design patterns can adjust. These three patterns can

be split further into class patterns and object patterns. The creational patterns make a system independent of how its objects are created, composed and represented [8]. Class creational patterns make user of inheritance to differ the class that's instantiated. Object creational patterns use other objects to instantiate themselves. Structural patterns are concerned with classes and objects are composed from larger structures [8]. Structural class patterns create implementations and interfaces using inheritance, whereas structural object patterns use objects to create new functionality. Behavioural patterns are concerned with algorithms and the assignment of responsibilities between objects [8]. This means that behavioural patterns describe the interactions between objects and classes. In behavioural class patterns, the behaviour of the classes is split between themselves with inheritance. Behavioural object patterns, on the other hand, use composition rather than inheritance.

One of the most widely used higher level patterns for programming languages is Model-View-Controller (MVC) [5]. MVC is used in programming to separate the different parts of the code into different layers. This means that the developers write the code to conform to how the layers interact with each other and what each layer should contain.

2.3.1 Model-View-Controller (MVC)

MVC is comprised of three layers, with each layer having its own specific function. The Model layer captures the logic of the system and the data it holds. The View layer presents the information from the Controller layer. The Controller layer handles the behaviour of the system by linking the user input with system input [5]. MVC requires the developer to split up the different aspects of the code into these layers. However, MVC allows for each developer to mould the pattern to their own individual style, whilst keeping the core elements the same. The Model layer, as mentioned above, captures the logic and the data used by the system. This means that the Model layer consists of mainly code that accesses the back-end database and code that stores any data needed for the database. The View layer handles all the code that outputs information to the user. The controller layer handles all the interactions between the view layer and the model layer.

2.4 Design Patterns and Principles and the Web

Design principles for web development are centred around making the system easy to use by the target audience, making the code structured and formatted, making the site as optimized as possible, making the site as accessible as possible and making the front-end design as responsive as possible. Design principles for the web allow the development team to create a checklist of qualities that the system should have, allowing them to create a functional system that follows the standards.

One of the patterns used to achieve this is the concept of Flexible Web-Application Partitioning (fwap) [6]. The fwap concept allows the user to utilize the MVC pattern, whilst making sure the web application can run wherever it is deployed. The fwap concept enables the developer to build web applications that are partition independent [6]. This means that if the application is deployed to different servers and clients, then the code will not need to be changed. This concept also implies that the partitioning could be changed without modifying the application [6]. The fwap concept allows for the developer to separate the code efficiently, whilst making sure that there is only one instance of the code within the

application. Figure 2.1 shows an illustration of how the MVC pattern looks within web application development.

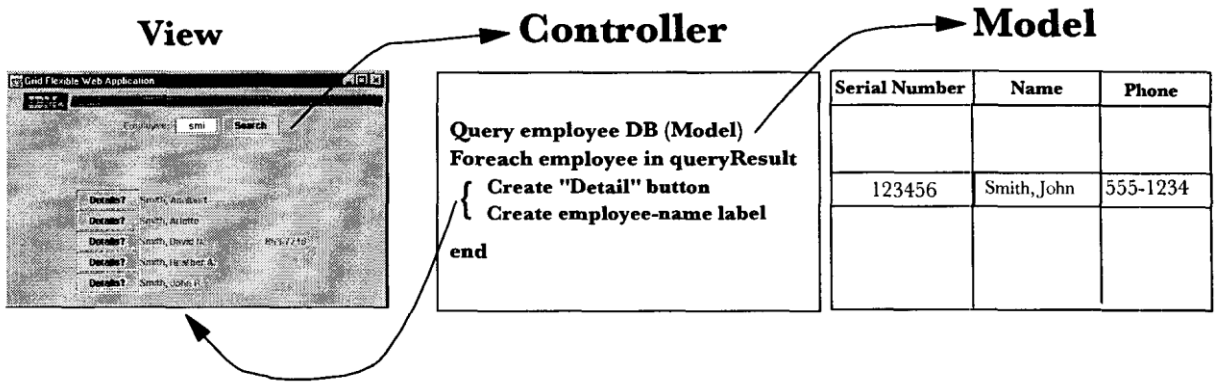


Figure 2.1 – MVC pattern in web application development [6]

MVC with fwap can be implemented in one of three ways. These are briefly outlined below. Each way controls how the partitioning in implemented in the MVC pattern will be distributed. The three ways of implementing mvc with fwap are the single-mvc (smvc), thin-client and dual-mvc (dmvc) architectures [8]. The smvc architecture involves the model, view and controllers execute and are stored in a single space and not separated between the client and server. The thin-client architecture involves storing and executing the model and controllers on the server, which then creates the view on the client side. The dmvc architecture allows for the model and controllers to be stored and executed on both the client and servers. Both the server and client can generate the view, which is then displayed on the client. These architectures are considered during the initial analysis, the design and the deployment phases of the software development cycle. This means that if the developers are using MVC with fwap, the type should be considered before the system has started to be coded.

2.5 Conclusion

Having read many sources on design principles and patterns, there seems to be recurring patterns across many different areas. These patterns correspond to the idea that the solution should be able to be used over and over again, with either changing nothing or changing everything. From this idea, there are many different patterns and principles that resonate through many different topics. These patterns and principles rarely change and, therefore, can be applied to web application development. Having read and discussed a few of the patterns and principles above, there are a few that stand out as being the most important to consider in web application development. These are listed below.

Pattern or principle	Summary
MVC	Model-View-Controller separation system for the files within an application. This includes Separation of Concerns. Meaning that the code is separated into different files, making it easier to read and easier to maintain. An example of separating concerns is a plain HTML page and then a separate stylesheet CSS page.
FWAP	This governs how the MVC will be deployed once the application has been developed.
Structural, Creational and Behavioural Patterns	These govern everything about the application and include several more specific patterns to help them achieve this. Not all the more specific patterns need to be used in order for these overall patterns to work.

From the most important patterns and principals above, the MVC pattern is the one that should be considered the most. This is because for a web application to be successful, the code needs to be well laid out, easy to understand and easy to change. MVC allows this to happen through the separation of the different files, and the separation of code with different functionalities. MVC allows for the re-use of code throughout the application, cutting down on the amount of code duplication.

3 Project Methodology

3.1 Project Methodologies

There are many different project methodologies available to use, each with their own advantages and disadvantages. There are many different methodology types: Traditional or Sequential, PMI/PMBOK, Agile, Change Management, Processes-Based and more. Each methodology type has their own subvariants. The most commonly used project methodologies are the waterfall methodology or an agile methodology like SCRUM. Below are some of the more common project methodologies used:

- Waterfall
- Critical Path Method (CPM)
- SCRUM
- Kanban
- Extreme Programming (XP)
- Event Chain Methodology (ECM)
- Lean
- PRINCE2

Project methodologies like Waterfall and Critical Path Method (CPM) are based upon following a strict linear path throughout the project lifecycle. This means that one task must be finished before the next task can begin. This is useful when the project has a clear definitive end, however, these methodologies become difficult to manage and costly if any changes occur. This is where the agile methodologies come in. Agile methodologies follow an iterative approach, meaning that they have a series of steps that are repeated like a cycle until the project is completed. This makes them adaptable and open to any changes. The Kanban methodology is a mixture of agile and sequential as each task is laid out on cards and put under the “to do” heading, these are then worked on and as each task is completed it is moved from “in Progress” to “completed”.

In addition to how each methodology deals with change, there is how each methodology copes with different project types. The project type has a direct effect on which methodology is used. For example, a software product may not have a definitive product will need a methodology open to change, where as a sofa building project will hand a definitive product so will use a linear methodology. The size of the project must also be taken into consideration, as some methodologies are more suited to large projects and others more suited to small projects.

3.2 Chosen Methodology

The methodology chosen for managing this project will be an Agile methodology. This is to allow for the requirements to change at any time. Using an agile methodology will allow the use of iterations to progress through the project, meaning that after one function has been implemented testing will be done on that function. This will ensure that there will always be a version of the system that is working and that any new features added will work with the current implementation. An agile approach allows

for the user and client to be involved throughout the whole project. This allows for immediate feedback and allows the user and client to see if the system is meeting expectations, meaning that the end artefact will be of a higher standard.

4 User Requirements Analysis

4.1 SWOT Analysis

A SWOT analysis is used to determine the Strengths, Weaknesses, Opportunities and Threats to the project. The strengths and weaknesses are internal to the project, whereas the opportunities and threats are external to the project. Internal strengths and weaknesses apply to the company or organization behind the project. These mainly involve factors surrounding the employees and system the company currently has. The external opportunities and threats apply to other companies or organizations with similar products or systems. The main factors affecting these are competitors with the same or similar project, similar products already available and external people or companies looking to harm the company or project. The SWOT analysis will help to determine whether the project is viable and worth putting into action by investigating all the internal and external factors relating to the project and evaluating the viability of the project from these. Figure 4.1 below shows the SWOT analysis.

Strengths	Weaknesses
Reduces Paperwork: Lecturers would not need to take attendance through paper based means, resulting in less human error and time saves in sessions. **Increased attendance: The system will encourage students to turn up so they do not fall into the Students At Risk (SARS) category.**	Data Error: The data entered may be incorrect or corrupted, causing attendance to be incorrectly stored. Single Developer: This may lead to a system without all the functions that users wanted and may lead to implemented functionalities taking longer to code.

Increased Time for lecturers: Lecturers will have more time in sessions for teaching and helping, and they will have more time in office hours to see students who need help.	Software: The system requires and internet connection to be accessed, if for any reason the internet is not accessible, the system will be unusable.
Opportunities	Treats
Lecturers: Lecturers who currently do not accurately keep track of attendance figures for session they teach will be able to easily keep track of this. **Other Academic Institutions: If the system delivers a high level of features and has a high satisfaction level then it could be released and melded to use by other academic institutions.**	Competition: There are other systems currently available by outside companies that may offer different functionalities. Unauthorized Access: People trying to break into the system and steal any sensitive data being stored or to insert malicious software or code into the system to make it fail. Different Solutions: There are other solutions available from outside sources that may be better suited to solving the problem this system is being designed to solve.

Figure 4.1 – SWOT Analysis

4.2 Conclusion

From the SWOT analysis above, the project is viable because whilst there are other systems of the same kind and systems that solve the problem in different ways, they are both costly to set up and install and costly to train people in how to maintain the system. Also, the system will benefit the users by reducing the time to take the register in sessions and by removing the need for lecturers to manually enter attendance data into spreadsheets/databases in office ours. This increases the time lecturers for teaching and helping in sessions and increases the time they have in office hours to have meetings with other lecturers and students. Overall this project proposes to be a more cost-effective option whilst still providing the features that the users want from the system.

4.3 Stakeholder Analysis

A stakeholder analysis is essential in the project planning stage. A stakeholder analysis identifies the people who influence the system, whether it be the end users or the business or team behind the project. There are three types of stakeholders. There are Key stakeholders, Primary stakeholders and Secondary stakeholders. Key stakeholders can have the largest influence on the project, and therefore, have the largest role. Primary Stakeholders have less influence on the project, but the project has either a positive or negative effect on them. Secondary stakeholders have the least amount of influence on the project and are indirectly affected by the system.

4.3.1 Key Stakeholders

Lecturers – The system directly affects the lecturers and how they operate. The system does this through allowing them to have an electronic view of which students are turning up to their lectures and workshops and identifying which are not. The lecturers are a major stakeholder of this project, and can have a major effect on what the system does and how it performs.

Head of University – The system will affect these stakeholders to a high extent. These stakeholders will want to use the system to see which courses have the highest and lowest attendance rates and to address any issues found. Whilst they may not use the system daily, they still have a high interest and influence on what characteristics the system has.

4.3.2 Primary Stakeholders

Students – Students will be the main users of the system, but will not have a high influence on the project. The students will be directly affected by the system and use the system daily.

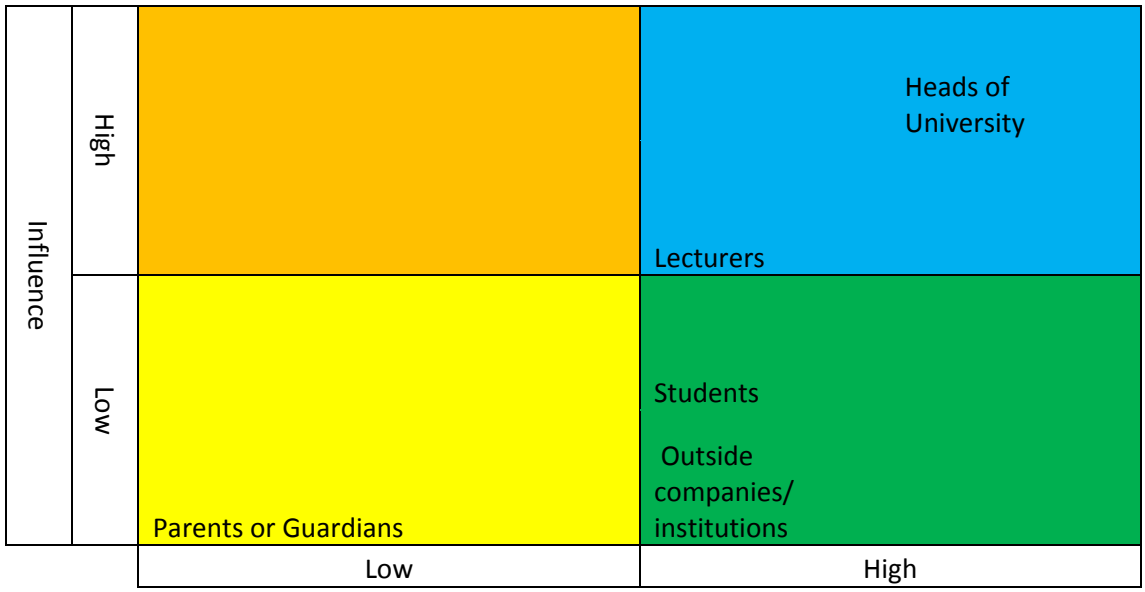
4.3.3 Secondary Stakeholders

Parents or Guardians of students – Whilst the parents and guardians would not directly access or be directly influenced by the system, they will have some interest in how the system works. They may also be interested in the outcome of the system as they will want to see figures on their child's attendance.

Outside companies/institutions – These will have a low influence but a high interest as they will want to know the attendance rates for potential job candidates. This will allow the company or institution to monitor who is turning up regularly and who is not.

4.3.4 Analysis

Figure 1 below shows the analysis of the stakeholders and how much influence and interest each stakeholder has.



Interest

Figure 4.2 – Stakeholder Analysis

4.3.5 Conclusion

The stakeholders for a project are an integral part of making sure that the project is successful. Therefore, each stakeholder must be considered when planning a project. From the analysis and figure 4.2 above, the main stakeholders are the lecturers and students as they are the main users of the system. There stakeholders will be included consistently along the project lifecycle. However, the heads of the university will also be taken into consideration, as they ultimately have the highest influence.

4.4 Requirements Analysis

To gauge what the users would like the system to do, user stories are created and analysed. These user stories outline a high-level goal that the user wants to achieve. These are then broken down into use cases which explore the steps needed to accomplish the goal. The use cases are analysed to establish the requirements of the system. The requirements are split into functional and non-functional requirements and a MoSCoW rule set used to determine which requirements have the highest priority and, therefore, must be in the system.

Below is a list of the user stories for each user of the system.

ID	As a…	I Want to…	So that…
1	Student	register for a lecture that is on my timetable	My attendance is recorded
2	Student	See my attendance for the week	I can see if I missed any sessions
3	Student	Change my password	No one else can gain access to my account
4	Administrator	Add a new student and give them a password	I can successfully add new students who join a course
5	Administrator	Add a new course and related modules	I can be up to date with the current courses offered by the university
6	Administrator	Update a course	Any modules related to the course will be up to date
7	Administrator	Remove a student	The student database

			is up-to-date and only holds the current students
8	Student	See my sessions for the coming week	I can make sure I can attend all sessions timetabled
9	Head	See all attendance for a course for a week for all sessions related to that course	I can see the overall attendance figures for all the sessions for a course
10	Head	Produce an attendance report for a course	I can see who is not attending the lectures and deal with any issues appropriately
11	Lecturer	See all attendance for a module I teach on for a week	I can monitor the attendance figures for the week and see what times are most popular
12	Lecturer	See any Students At Risk(SARS)	I can identify which students need more help or which students need to be contacted

Functional requirements define what functionality the system should have, as defined by the users. Non-functional requirements are functions of the system that define the performance of the system. The functional requirements will be defined below, then a MoSCoW analysis done on those requirements.

4.4.1 User Story Cards

User story cards are used to elaborate on the high-level user stories. The story cards allow the user stories to be broken down into several smaller tasks that need to be completed before the user story can be accepted as accomplished. The story card also allows any ore-conditions to be laid out, meaning that anything that must be completed before the tasks can be undertaken is identified and complete. The comments allow for the user story to be fleshed out and explained to gain a better understanding of what the tasks, including the pre-conditions, require and what they will accomplish.

Below are some of the story cards for the user stories above, see figures 4.3, 4.4 and 4.5. The remainder of the user stories can be found in the appendix, see appendix X.

Story ID	**1**

As a	Student
I want to	Register my attendance at a session I am timetabled for.
So that	My attendance is recorded
Pre-Conditions	Student Registered on system Student correctly logged in
Acceptance Criteria	Student able to see and click a register button for the session they are in. Student clicks button and enters their login details again. Student sees a confirmation message.
Comments	Students logged in again to ensure they are in the session. This log in will require a specific password set up by the lecturer in the session.

Figure 4.3

Story ID	**10**
As a	Head
I want to	Produce an attendance report for a course
So that	I can monitor the attendance figures for the week and see what times are most popular
Pre-Conditions	Correct log in details entered. Correct session type.
Acceptance Criteria	Head can see an online table containing the figures for a chosen week. Head can change the week to see different weeks. Head can see a button and clicks it to produce a report for the week.
Comments	

Figure 4.4

Story ID	12
As a	Lecturer
I want to	See any Students At Risk(SARS)
So that	I can identify which students need more help or which students need to be contacted
Pre-Conditions	Correct log in details entered. Correct session type.
Acceptance Criteria	Students correctly assigned to SARS status if certain conditions are met. Lecturer correctly sees the SARS Students. Lecturer able to create a report of all students who fall under SARS status.
Comments	

Figure 4.5

4.4.2 Functional Requirements

- Login – Any registered user of the system should be able to enter a unique identification number and a password into a login page, which will grant access to the system.
- Logout – Any user who has successfully logged in should be able to logout of the system when finished.
- Restrict access – The system should restrict access to certain functions and features to certain user types. The system should be able to detect the type of user to accomplish this.
- Change password – A user should be able to change their password if they require,
- Register attendance to a session – A registered user should be able to register their attendance at a session they attend.
- View class attendance – A registered user should be able to view their own attendance for their scheduled classes.
- View module attendance – A lecturer and head should be able to view attendance figures for their own modules and other modules.
- View course attendance – A head should be able to view overall attendance figures for a course of their choosing.
- Calculate attendance percentages – The system should be able to calculate the percentage of attendance for sessions, modules and courses.
- Produce reports – Lecturers and heads should be able to create a report detailing the attendance figures for modules and courses.
- Add and delete users – The system administrator should be able to add and remove users from the system.

- Add and delete course and modules – The administrator should be able to add new courses and modules to the system and register students to them.

4.4.3 Non-Functional Requirements

- Accessibility – the system should be easy to navigate around and the functionalities easy to use.
- Platform – The system should be accessible from a laptop/PC.
- Operating System and Browser - The system should be accessible from the most popular operating systems and browsers.
- Response times – The system should respond quickly to whatever request is made of it.
- Maintainability – The system should be easily maintainable by anyone, without having to train them in anything.
- Reliability – The system should be up and available to users at least 90% of the academic year.

4.4.4 MOSCOW

A MoSCoW analysis is used to determine the importance of the requirements identified above. This will determine which of the functions the system will have and which it might not have.

Must Have:

- Login for all users to make the system secure and accessible to only those who are registered.
- Enable students to mark their attendance at each session, so the correct data is stored in the database.
- Lecturers should be able to view the attendance figures for sessions they teach.
- Users must be able to change their own passwords.
- Administrators must be able to add and delete users from the system.

Should Have:

- Heads should be able to see overall figures for a course of their choosing.
- Students should be able to see an overall percentage figure for their own attendance.
- Students should be able to see their attendance records for each scheduled session.
- Reports should be able to be produced on a session, module and course basis by heads.
- Functionality to be accessed on mobile devices.

Could Have:

- An automatic alert system for Students At Risk (SARS). These are students that all under a set percentage of attendance.

- An automatic email sent to students who fall into the SARS category.

Would Have:

- Ability to turn into a stand-alone application.
- Ability to record coursework, in-class test and exam results and monitor grades for students.
- Module feedback form.

5 Design

This chapter will discuss all aspects of the design phase. It will display and discuss the techniques used to help design the system. The discussion and analysis of the requirements in the previous chapter will be used to create some conceptual designs for the system. The conceptual designs will be used as a starting block for implementing the system, which will be discussed in the following chapter.

The first aspect is the design of the database. This is potentially the most important aspect as the database will hold all the information and govern how the front-end site will handle information. The designing of the database consists of a class diagram, Activity diagrams and an Entity Relationship Diagram (ERD). These designs will be used to create the database and how the front-end will interact with it.

The Second aspect is the design of the web pages. This will govern how the site will look to the user and what elements on the site can interact with the database. This is an important step as factors including accessibility, usability and security should be considered at this stage.

The aims of the project are to allow students to record their attendance at sessions online and remove the need for a paper based registration system, allow the lecturers to have a clear and easily manageable electronic view of attendance figures for each course, module and session they teach on, to cut down on the number of students registering their friends as attending when they are not there and to provide a secure environment for session attendance to be taken. These aims encompass some of the ideal outcomes from the system, however, there are several outcomes that could be added such as: having a timetable system that shows up on a calendar, with the sessions marked in green if attended and red if not. Therefore, the design chapter sets up what the system will do, how it will do it and how it will look to the end user.

5.1 Database Design

5.1.1 Activity Diagram

Activity Diagrams show how the user will interact with the system for a specific task. This allows the creator of the system to see what interactions take place and how they take place. The diagrams break down key tasks and show all steps within the task. Figure 5.1 shows the student view for their attendance, and being able to choose whether they want to view an overall attendance or view it by sessions in a module.

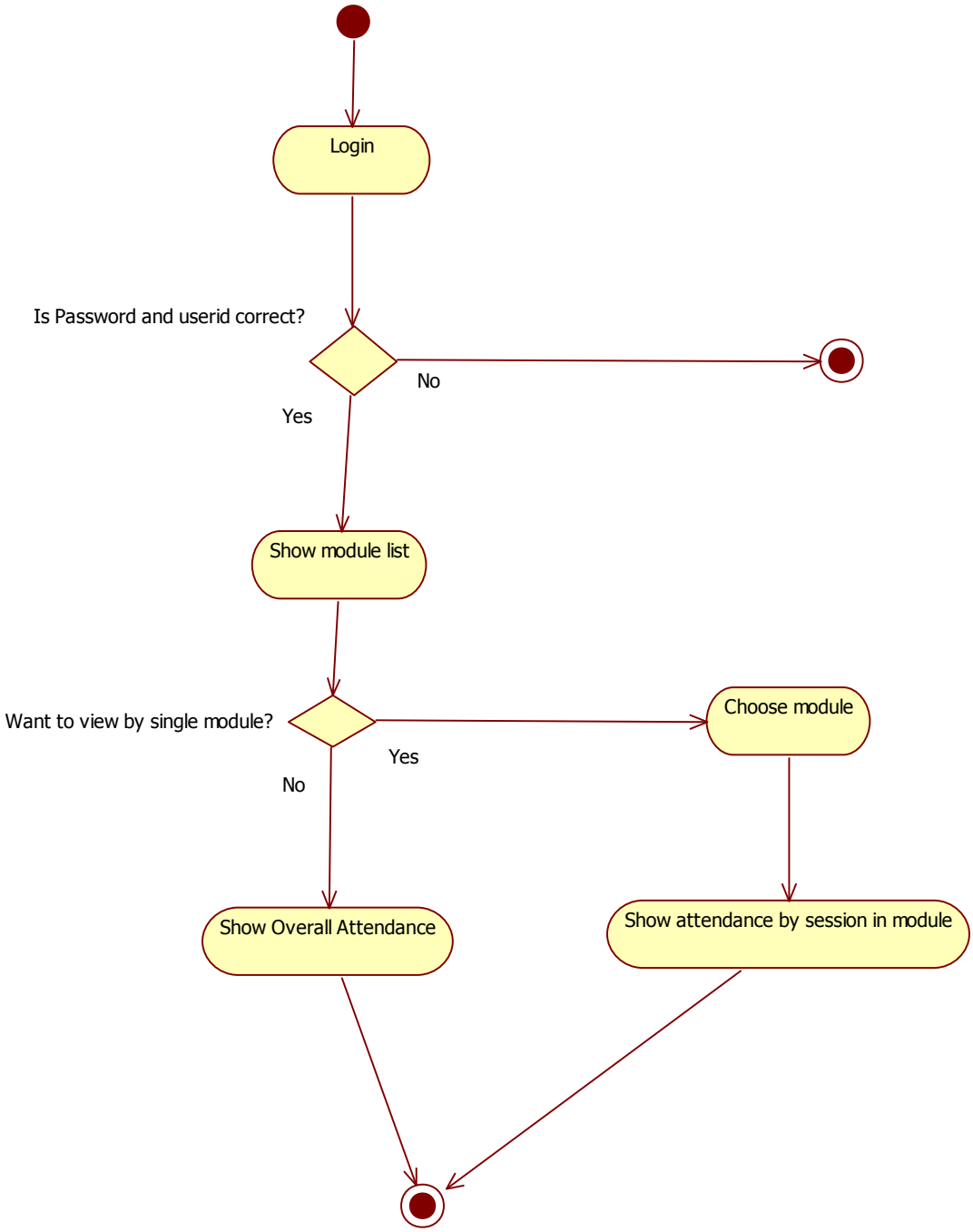


Figure 5.1 Activity Diagram – Student view attendance

5.1.2 Class Diagram

Class diagrams show the main classes or objects that make up the system. This includes the relationships between the classes. Class diagrams are used to give a view of how the database will look and how each table will be linked. Figure 5.2 shows the class diagram for the online student attendance system

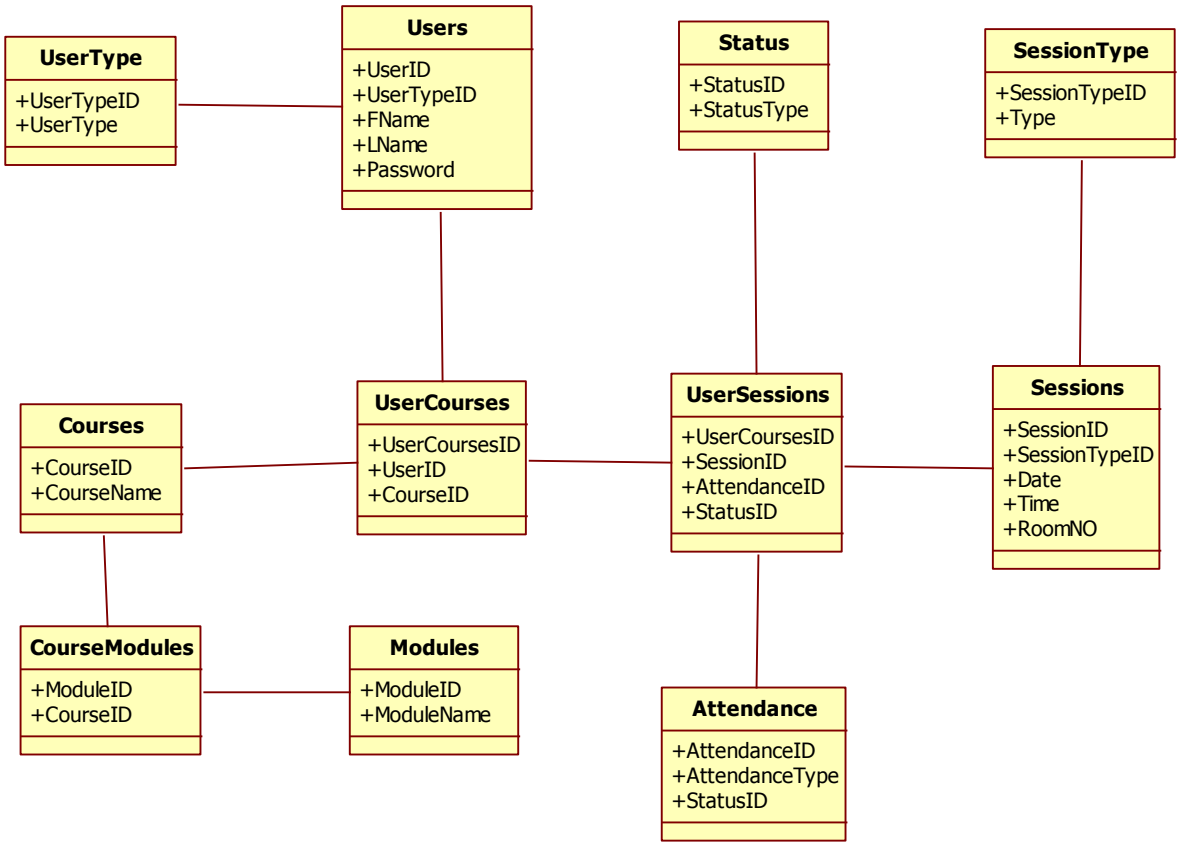


Figure 5.2 Class Diagram

5.1.3 Entity Relationship Diagram (ERD)

Figure 5.3 is an ERD. An ERD is a graphical representation of a system that shows the relationships between the entities of the system. This is a data modelling technique that can be used to create a foundation for a relational database to be created.

An ERD shows the tables, attributes in the tables and the links between the tables, defining the cardinalities. For example, “1” is only one and “1..*” is one or more. The cardinality represents how many of an object another object can have. For example, in Figure 5.2 above a user has 1 or more modules and a module has 1 or more users enrolled on it. The table below has a brief description of each table in the ERD and any key relationships the table has to the other tables.

Table Name	Description	Key Relationships
UserType	Defines the type of user accessing the system. This is used to control the sessions and permissions of the system.	Each user can only have one user type.
Users	Used to model all the users of the system. This models all the main attributes of a user.	Links to the usertype table so that the system knows what type of user each user is.
UserModules	Holds the information about which user is on which modules	Used to link the users to modules because the relationship is many to many. This means that when the table is accessed, there will only be one instance of a particular student on a particular module.
Modules	This table models all the modules for the courses.	
CourseModules	Holds the information about which modules belong to which course or courses.	This table links the modules with the courses so that there will only be a single instance of a module with a course.
Courses	This table models all the courses available.	
SessionType	This table monitors the type of sessions available.	This links to the sessions table to control which session is of which type.
Sessions	This table holds all the information about a specific session.	
Status	This table holds information about a particular status of a user if there is one. This means that the user may not have a particular reason or status for not attending.	This links to the sessions table to monitor if the user has not attended but has given a reason.
Attendance	This table is used to hold the information about if a user has attended a session or not.	This is linked to the sessions table to make sure that a particular session has its attendance stored.

.

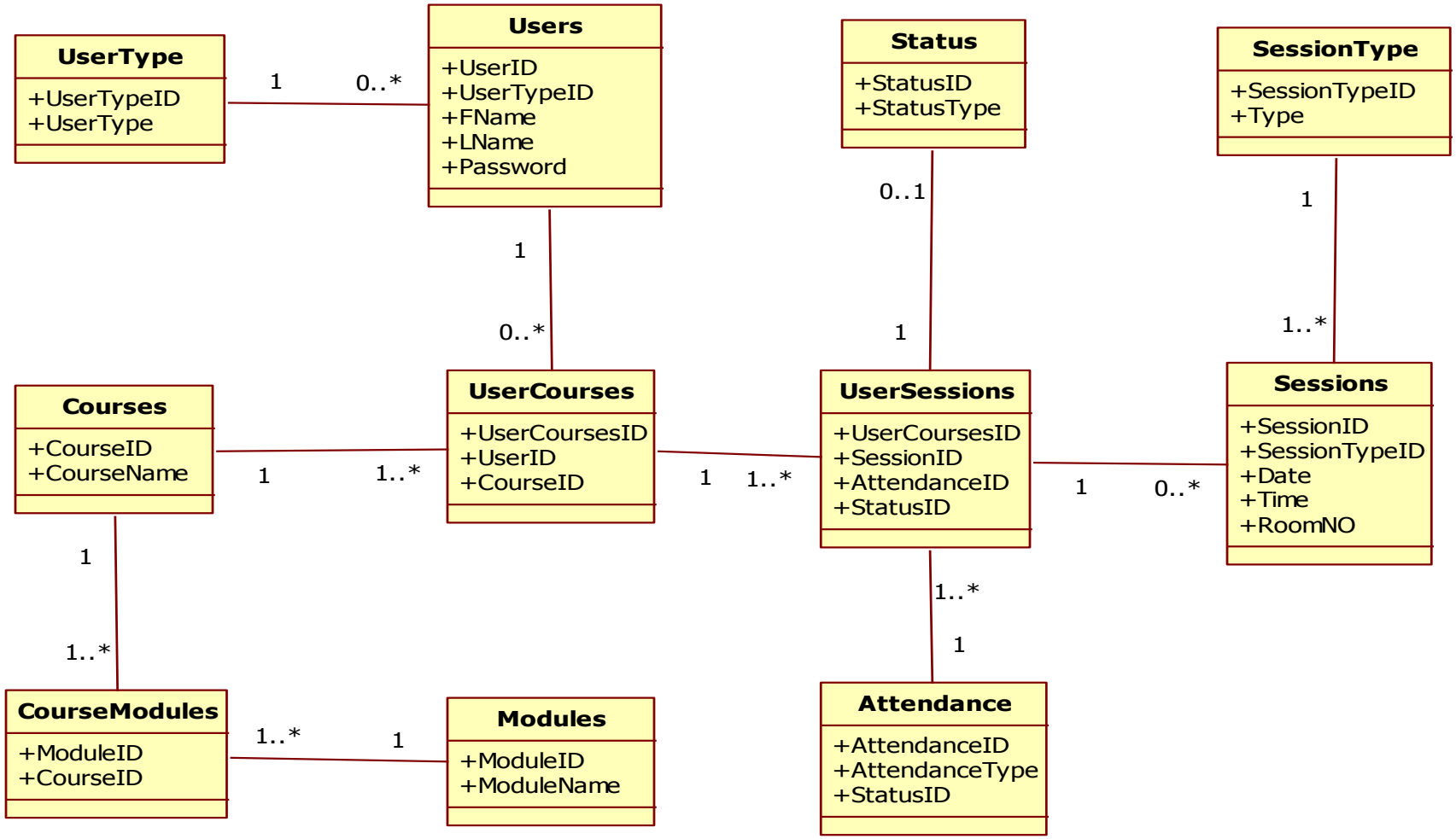


Figure 5.3 ERD Diagram

5.1.4 Data Dictionary

Figure 5.4 and Figure5.5 show the details of all the attributes included in the database, including the type of data stored, whether it must have a value in it and any foreign key references. The notes section defines any other key information like the encryption or the format of the data

Table Name	Attribute Name	Description	Data Type	Range	Required	References	Notes
Sessions	SessionID	The sessions ID	Int (11)				
	SessionTypeID	ID of session Type	Int (5)		Y	SessionType	
	Date	Date of the Session	Date				
	Time	Time of the session	Varchar (5)				
	RoomNO	The no of room of the system	Varchar (8)				

Figure 5.4 Data Dictionary – Sessions Table

Figure 5.4 shows the sessions table from the database. This is the table that will hold the details about a specific session, with the type of session being stored as an id from another table. The time uses the varchar data type over the timestamp data type because the varchar datatype is easier have many different time formats available and is easier to manipulate than the timestamp data.

Table Name	Attribute Name	Description	Data Type	Range	Required	References	Notes
Users	UserID	The users ID	Varchar (8)		Y		Kxxxxxxx
	UserTypeID	ID of user Type	Int (11)		Y	UserType	
	Fname	The users' forename	Varchar (15)				
	Lname	The users' surename	Varchar (15)				
	Password	The users' password	Varchar (20)				Encrypted by database

Figure 5.5 Data Dictionary – Users Table

Figure 5.5 shows the users table for the database. This is where all the data for each user is stored. This is also where the users' login details will be kept. The password for each user will be encrypted so that the security of the system can be maintained. This will be done through hashing and salting algorithms. The user id will be controlled so that it will be in a specific form, KXXXXXXX. This constraint will be used to control what the user can enter into the user id field.

5.2 Web Interface Design

5.2.1 Design Principles and patterns

When designing the pages for the site, the design principles and patterns discussed in chapter 2 must be considered. These principles will be used as a guideline to define how the website will look and how it will be interacted with. Whilst all design principles are considered in chapter 2, this chapter is mostly focused on the patterns and principles surrounding web application design.

Many sources consider there are 10 main principles to web design: purpose, communication, typefaces, colours, images, navigation, layout, "F" pattern design, responsiveness and platform compatibility. The purpose of these principles is to guide the design and implementation to create the best web application possible. Below is a brief description about what each principal defines.

Principal	Description
Purpose	This defines what the website is about by analysing the needs it the user and applying them to specific pages.
Communication	This defines how the information on the site is displayed to the user. I.e. making the information easy to read and digest by the user and separating the text using headings.
Typefaces	This defines the font that the website will use, including the size of the text and how many different fonts are acceptable.
Colours	This defines the colour scheme for the site to enhance the user experience.
Images	This defines choosing the right images for the site and using the images in the right way to make help the user not hinder them.
Navigation	This defines how easy the website is to move around. This covers page hierarchy, bread crumbs, the 'three click rule' etc.
Layout	This defines the overall layout of the pages in the site.
"F" Pattern design	This defines how the user is most likely to look at the computer screen, and therefore, which parts of the site are they most likely to read.
Responsiveness	This defines the amount of time each page in the site should take to load.

Platform Compatibility	This defines whether the website is compatible with different platforms, such as mobile, tablet etc.

5.3 Conclusion

In conclusion, these principles are used to guide web development to create a web application that is both functional and useable by the intended audience. This means that no matter what the site content is or where the user is viewing the site from, the site will function as expected and provide a suitable service for each user. Therefore, it is important to take the principles into consideration throughout the design and implementation phases of the system life cycle.

The principles outlined above will guide the design and implementation of this web application. The purpose has been split into the different user stories and described in the chapter above, the other principles will be outlined in the rest of this chapter.

5.3.1 Sketches and Storyboard

Figure 5.5 below shows the overall map of the website, inclusion showing how each page or part is accessed and who can access it. This is done through a layered tree diagram so that the paths to each page or area can clearly be shown.

Figure 5.5 displays the different paths through the site for the different users. The different paths represent the pages that the user may visit after seeing their home page.

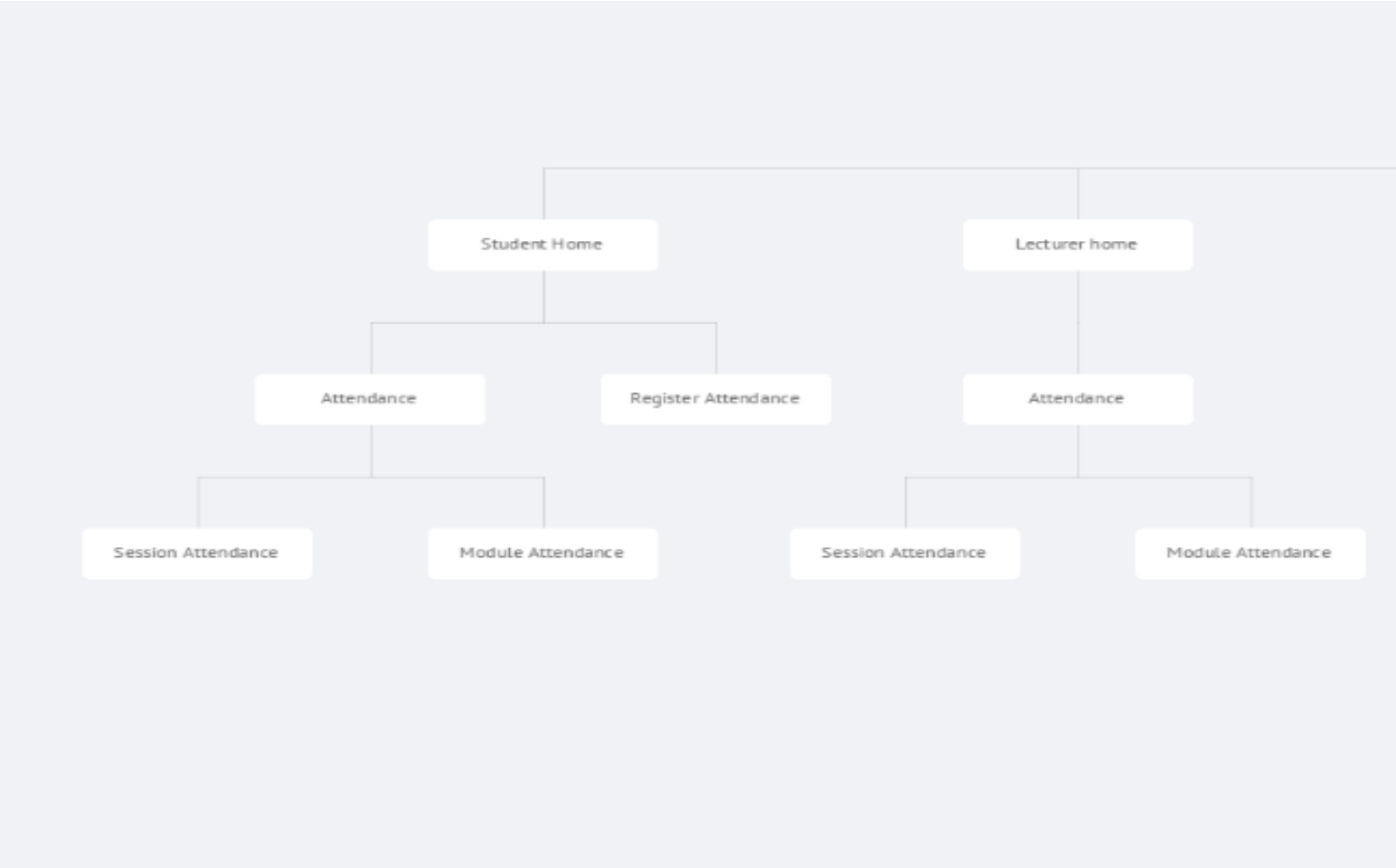


Figure 5.5 Website Map

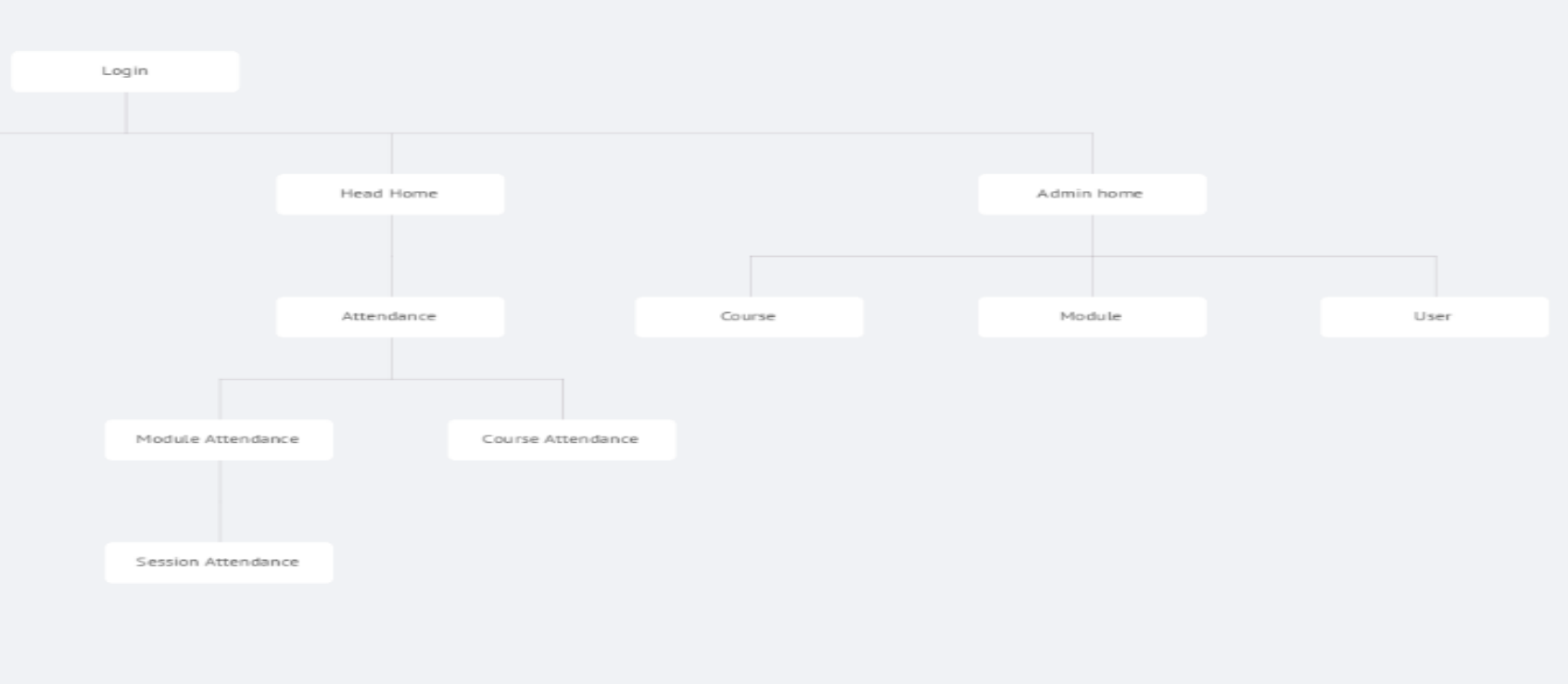


Figure 5.5 Website Map

5.3.2 Wireframes

Below are some of the wireframes created to display a prototype of what the page layout might look like. Wireframes are used to plan the designs and layouts of the pages; however, these wireframes are only a preliminary plan and the final design and layout of the implemented pages may be different. Figures 5.6 to 5.11 show the preliminary wireframes for some of the pages within the online student attendance system.

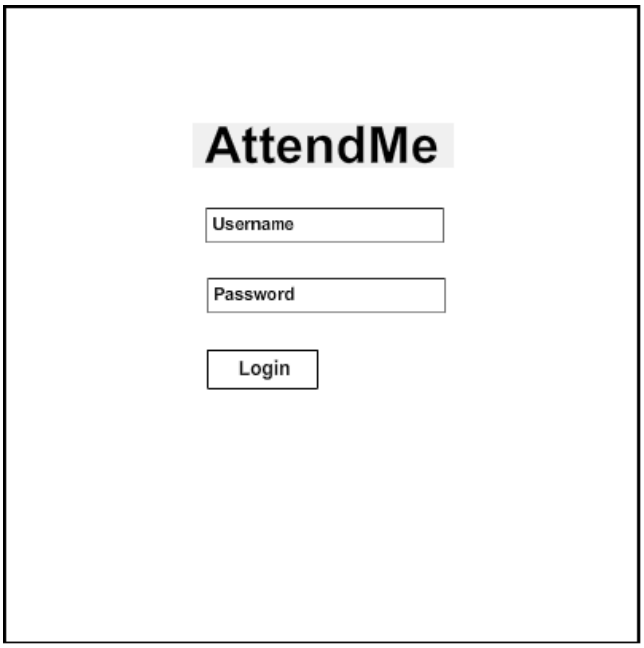


Figure 5.6 Login Page

The Login page will simply consist of a login area in the middle of the page, with the site name and logo situated above input area.

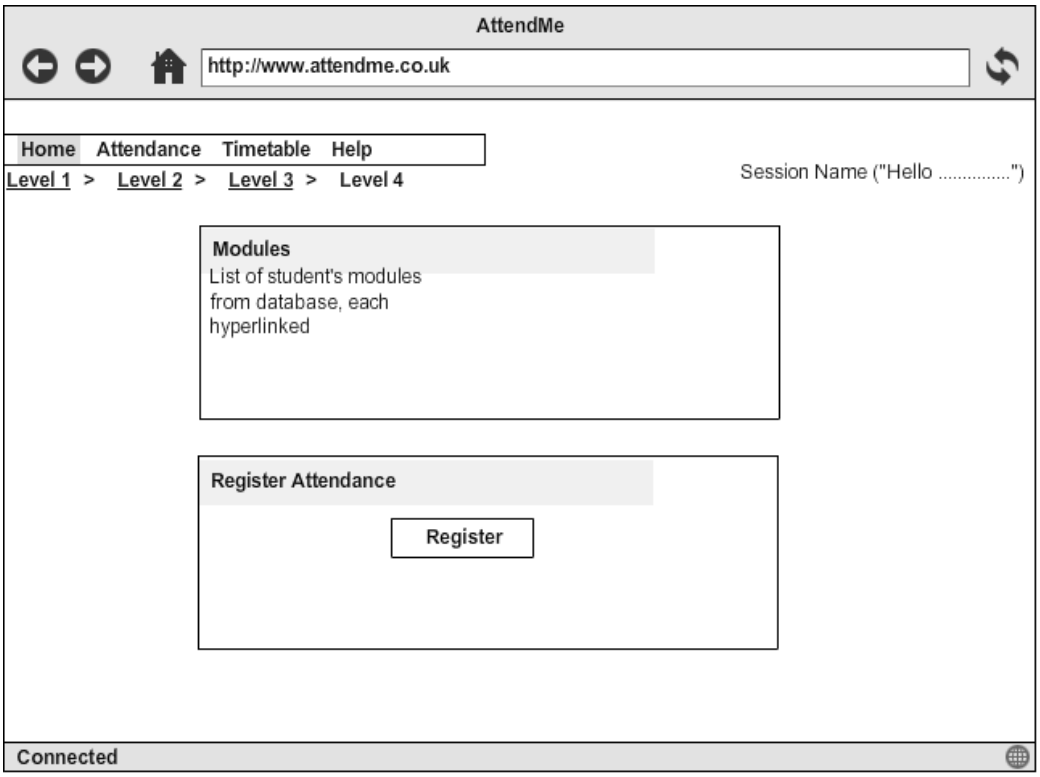


Figure 5.7 Student Home

The Student home page has several features that will be the same over all the other pages, except for the login page. These include the navigation bar at the top of the page, the session name in the top right hand corner of the page, the “breadcrumb” trail at the top below the navigation bar. The two boxes in the middle of the page will be used to separate the different sections. The top section will show the users modules, grabbing the list using the session. The bottom section will display a link that will open a pop-up page for the user to enter their login details, registering them for the session.

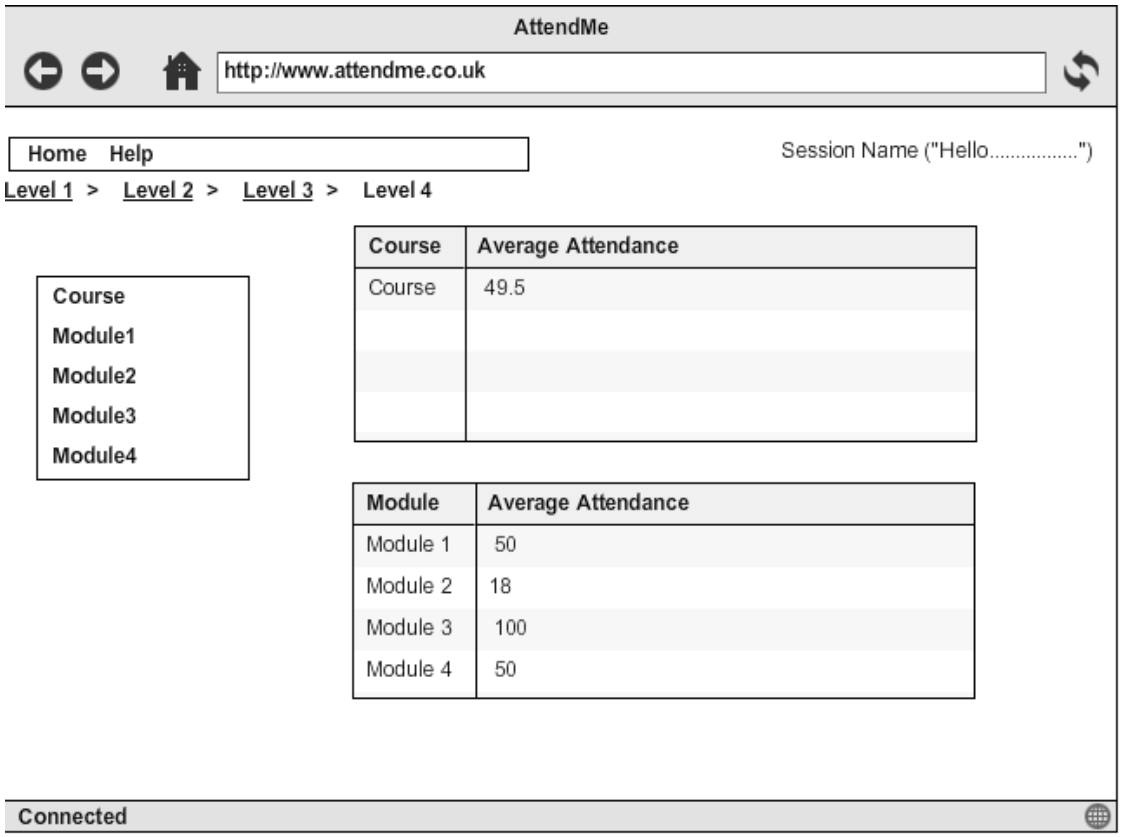


Figure 5.8 Student View Attendance

The list on the left-hand side of the screen allows the boxes on the right-hand side shows the users course and modules, used to take the student to a page showing a single box in the centre of the screen with the attendance figures on it for that module. The boxes in the middle will by default show the average attendance for the user by the course and the average attendance for each module.

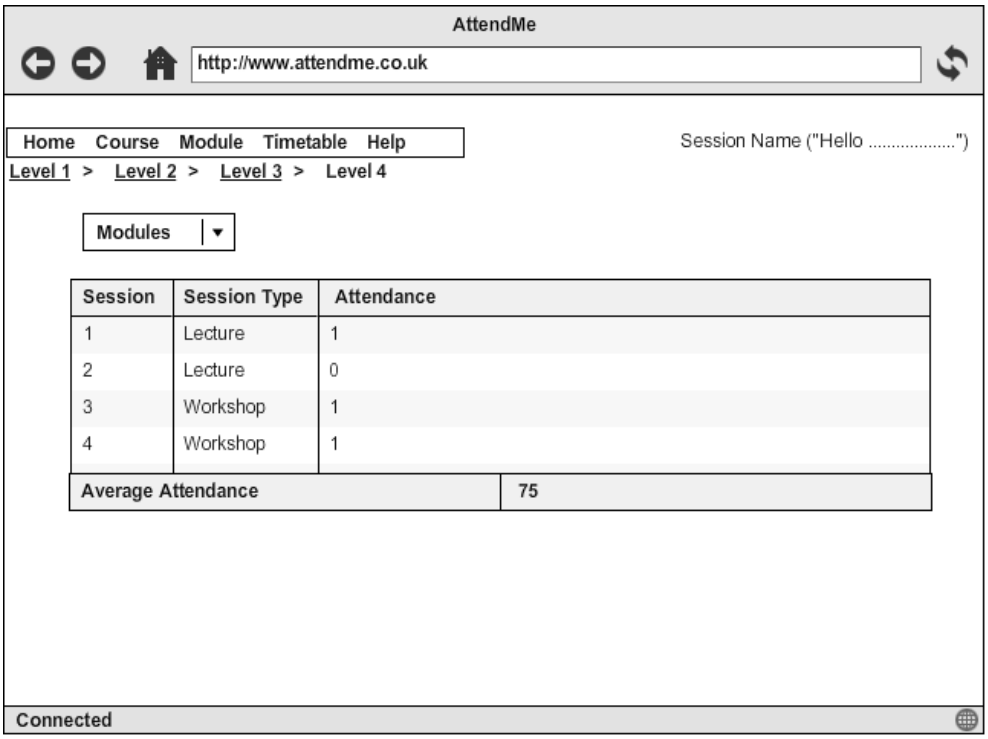


Figure 5.9 Student View Attendance by Module

This allows the student to view their attendance for each module viewing the attendance for each session. The user can change which module they want to view by selecting it drop-down menu.

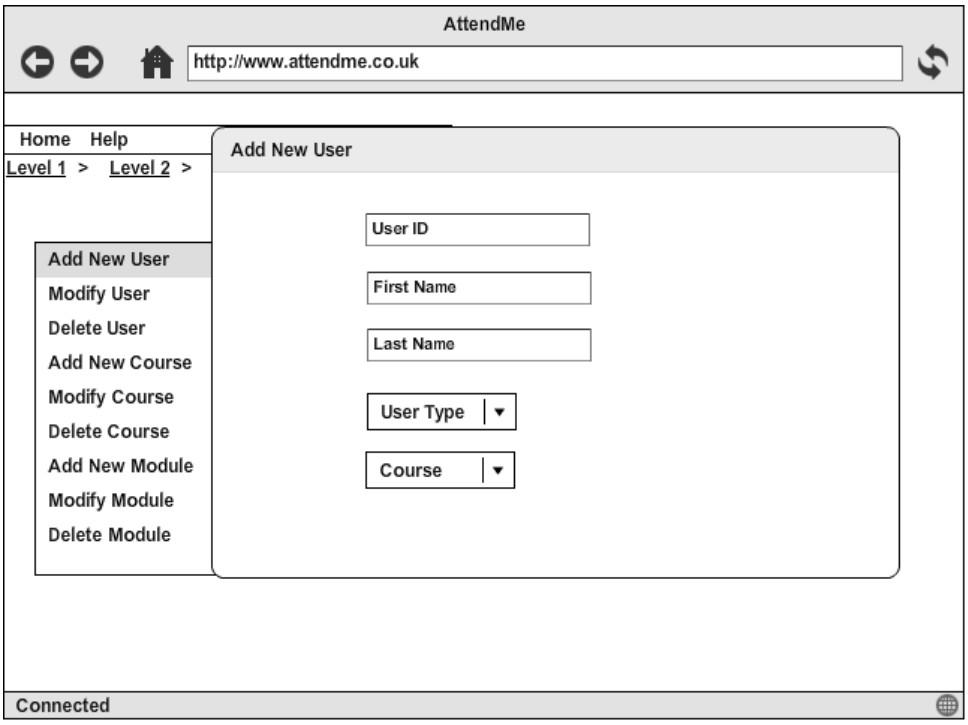


Figure 5.10 Admin Home, with Add New User option

This page is the home page for the administrators for the system. The buttons down the left-hand side will either open pop-up windows or hide and unhide boxes on the page to give the effect of a window system. This is modelled here as a window system to give the desired effect.

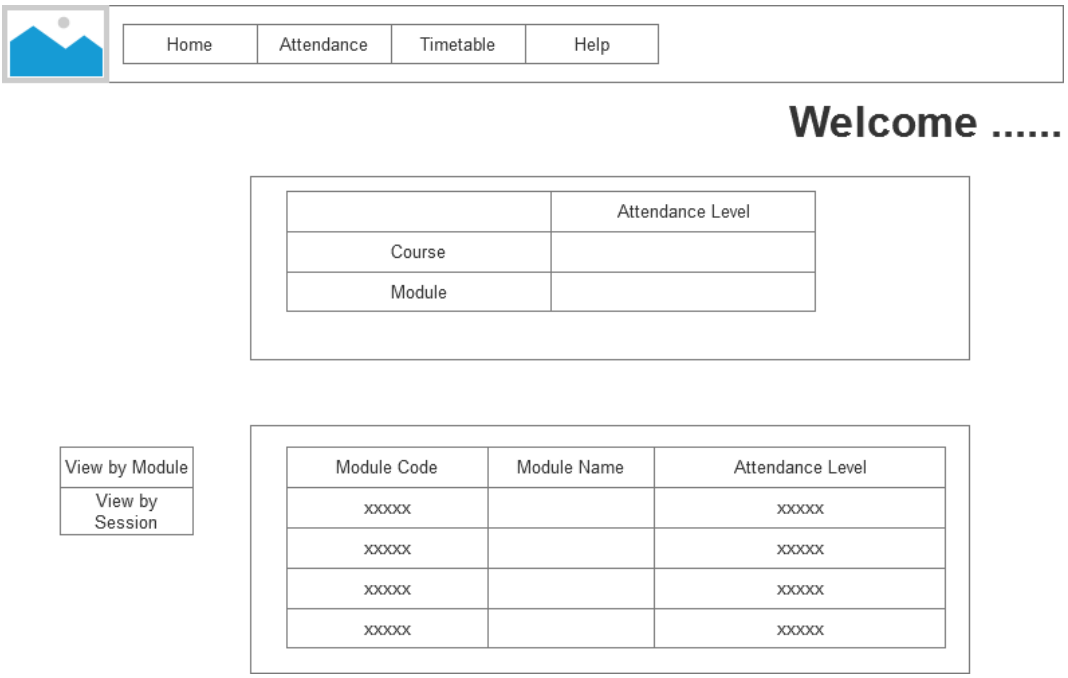


Figure 5.11 Updated Student View Attendance

In this updated wireframe for the students view of their attendance, the logo has been added to the navigation bar at the top of the page. Also, a little more style has been added, with the options on the left-hand side becoming more of a button system.

5.4 Conclusions

From the discussion above, the designs will help to create a website that conforms to the user requirements of accessibility, reliability, usability and security. The chapter above will help to form a guideline for implementing the system.

6 Implementation

This chapter will discuss the coding of the web application and the database to support it. To accomplish this, the technologies and languages used to build the system will be investigated and discussed, the methods and techniques used to code the website and the steps taken to build each page. The chapter will contain code extracts to show how the website was built, with the code extracts being explained and decoded.

6.1 Technologies and languages

6.1.1 Technologies

There are many technologies and languages available today which could be used to implement this project. The available technologies vary from free technologies to paid subscriptions and paid licenses. Each technology has its advantages and disadvantages and these must be considered when choosing a platform from which to implement the system. Figure 6.1 below shows some of the available technologies and their advantages and disadvantages.

Technology	Utility	Advantages	Disadvantages
PHPStorm	Text Editor	❖ Free (With university account) ❖ Colour coding on code ❖ Auto complete ❖ Highlights incorrect code ❖ Hints when writing code	❖ Only free for a period
Notepad++	Text Editor	❖ Free ❖ Widely available ❖ Highlights incorrect code ❖	❖ Not colour coded
Dreamweaver	Website Builder	❖ Allows for visual design along side	❖ Paid license ❖
WordPad/Notepad	Text Editor	❖ Free ❖ Cones pre-installed on devices	❖ Very basic ❖ No colour coding ❖ No formatting ❖ No code hints

PHPMyAdmin	Database Manager	❖ Free ❖ Widely used ❖ Wide support	
StudentNet	Hosting	❖ Free ❖ University Support	❖ Unreliable ❖ Needs a file transfer program
External Hosting (Godaddy, siteground)	Hosting	❖ Fairly reliable ❖ Comes with own upload systems	❖ Paid subscription ❖ Outside support
OracleSQL Developer	Database Manager	❖ Sophisticated systems	❖ Paid License

Of the technologies above, PHPStorm, PHPMyAdmin, and outside company hosting have been chosen to implement this project. They were chosen because they represent the best choices of the ones considered; PHPStorm because it has the best features and is free for students, PHPMyAdmin because it is free and had lost of support readily available, external hosting because even though it is no free, it provides the best solution as it is reliable. PHPMyAdmin was chosen over Oracle SQL as it is free and is widely used for smaller projects. Hosting was chosen to be acquired from an external company as the Studentnet hosting is unreliable. However, the external hosting costs, but it is more reliable and has more features and support than Studentnet hosting. There has been some thought and consideration into suitable backups for the chosen technologies as there may be times when they are unavailable. Therefore, suitable contingencies have been considered. Notepad++ has been chosen as a contingency for PHPStorm. Notepad++ has been chosen as the most suitable contingency because it provides the best features, is free and is widely available. As a contingency for the external hosting being unavailable, the project could be run from a localhost.

6.1.2 Languages

Web applications vary in the languages they use. These languages add various features to web applications, from a link to a back-end database to adding functions that allow the web pages to automatically update.

Languages	Utility	Advantages	Disadvantages

PHP	Front-end to back-end link	❖ Easy to use ❖ Prior knowledge ❖ Industry standard	❖
HTML	Web page code	❖ Easy to use ❖ Prior knowledge ❖ Industry standard	❖
CSS	Web page styler	❖ Easy to use ❖ Prior knowledge ❖ Industry standard	❖
Javascript	Web functions	❖ Easy to use ❖ Industry standard	❖ Some prior knowledge
AJAX	Asynchronous Web Code	❖ Allows for easy interactive web applications	❖ Requires teaching from Scratch
MySQL	Database Development	❖ Easy to use ❖ Prior knowledge ❖ Industry standard ❖ Large support ❖ free	❖
OracleSQL	Database Development	❖ Large Support ❖ Can handle large databases easily	❖ Paid subscription
JSON	Front-end to back-end link	❖ Easy to read ❖ Easy to understand	❖ Requires teaching from scratch

JSon, OracleSQL and Ajax are the languages above that were not chosen to implement the project. The main reasons for this is they would require learning from scratch or require a paid subscription or license to use them, increase the time taken to complete the project or the cost. This would have meant that the project might not have been completed on time. The other languages were chosen because not only would they require less learning, they are widely considered the standard languages for creating web applications and the database behind it. The technical support for these languages is good and widely available.

6.2 Database Creation

The database was created using the designs from the previous chapter using PHPmyAdmin. Firstly, the database was created and given the name attendme_db, making sure that the name was easy to relate to the project.

Secondly, the database was filled with tables. This was done in a variety of ways. Initially, they were created using SQL create table statements, however, after exploring the technology, an interface that created the table itself was found. This was then used to create the tables, but SQL was then used to create any constraints and links between the tables. After more exploring of the technology and learning, the function that allowed for any constraints and links to be made was found and this was used on the last tables in the database. The tables were created in a logical order, starting from the “outside” tables (tables with no foreign keys) then moving through to the “inside” tables (tables with one or more foreign keys). This meant that all the tables could be created and links set up immediately.

The last stage of the database creation was to input some “dummy” data into the tables. This would ensure that all the tables were correctly set up and any links and constraints worked as expected.

6.3 Interface Creation

The interface was created using a combination of PHP files, some containing PHP code and HTML code and some containing pure PHP code, CSS files and some Javascript files. All these files link together to create the interface that the user sees and accesses. The CSS stylesheets are used to control how the HTML pages look, with each page of the site having its own CSS stylesheet. However, as there is a common theme to the website, the pages do have a common CSS page.

6.3.1 Index.php

```php
<?php
    include_once "Model/db_connect.php";

    session_start();
    $_SESSION['username'] = $_POST['username'];
    $_SESSION['password'] = $_POST['password'];

    $sql = "SELECT * FROM users WHERE UserID='$_SESSION[username]' AND Password='$_SESSION[password]'";
    $result = mysqli_query($conn, $sql);
    $numrows = mysqli_num_rows($result);

    // Checks the type of user
    if ($numrows > 0) {
        while($row = mysqli_fetch_array($result)) {
            $UserType = $row[2];
            if ($UserType == '2') {
                header("Location:View/student_home.php");
                die();
            } else if ($UserType == '1') {
                header("Location:View/lecturer_home.php");
                die();
            } else if ($UserType == '4') {
                header("Location:View/admin_home.php");
                die();
            }
        }
    // Redirects to logout.php to clear session which in turn redirects back to login page
    } else {
        header("Location:View/logout.php");
        die();
    }
```

Figure 6.1 Index.php

The index page is the first page that will load when the user accessed the site. The code here sends the user straight to the “login.php” page to allow them to enter their credentials. The index can also send the user to the “logout.php” page which clears and resets the session, allowing the user to safely log out completely. The index page also handles the redirection for the login, sending the user to a specific page depending on what type of user they are.

6.3.2 Login.php

```php
            </ul>
        </div>
        <!-- /.navbar-collapse -->
    </div>
    <!-- /.container -->
</nav>

<!-- Page Content -->
<div class="container">
    <div class="row">
        <div class="col-xs-2 col-sm-4 col-md-4 col-lg-4"></div>
        <div class="col-xs-8 col-sm-4 col-md-4 col-lg-4">
            <div class="login">
                <div class="login-triangle"></div>
                <h2 class="login-header">Log in</h2>
                <img src="../Images/attendmelogo.png" alt="AttendMe" width="400" height="199">
                <form class="login-container" action="../index.php" method="POST">
                    <p><input name="username"  type="text" id="username" placeholder="Username"></p>
                    <p><input name="password" type="password" id="password" placeholder="Password"></p>
                    <p><input name="submit"  type="submit" value="Login" ></p>
                </form>
            </div>
        </div>
        <div class="col-xs-2 col-sm-4 col-md-4 col-lg-4"></div>
    </div>
</div>
<!-- /.container -->

<!-- jQuery -->
<script src="https://ajax.googleapis.com/ajax/libs/jquery/2.2.0/jquery.min.js"></script>

<!-- Custom JS -->
<script type="text/Javascript" src="../Javascript/custom.js"></script>
```

Figure 6.2 login.php

The login page is a simple page in terms of code. The user sees a form with two text boxes, one for their user name and the other for their password. They press the “submit” button that sends the information they entered to the “index.php” page that then identifies whether the information is correct or not. If it is the user is directed to the relevant home page, if it is not however then the user is directed back to the login page.

6.3.3 Admin_home.php, Student_home.php, Lecturer_home.php

```php
<div class="form-group">
    <label for="id">ID Number:</label>
    <input type="text" class="form-control" name="id" value="<?php print $id; ?>" readonly="readonly">
</div>

<div class="form-group">
  <label for="kid">User ID:</label>
  <input type="text" class="form-control" name="kid" placeholder="Enter an ID" maxlength="8">
</div>

<div class="form-group">
  <label for="FName">First name:</label>
  <input type="text" class="form-control" name="FName" placeholder="Enter first name" maxlength="50">
</div>

<div class="form-group">
  <label for="LName">Last name:</label>
  <input type="text" class="form-control" name="LName" placeholder="Enter last name" maxlength="50">
</div>

<label for="Course">Course:</label>
<select class="form-control" name="Course">
    <option value="" disabled selected>Please select a course...</option>
    <?php
        // Gets all the courses from the database to display in the dropdown
        $sqlcourses = "SELECT CourseName FROM courses ORDER BY CourseID";
        $resultcourses = mysqli_query($conn, $sqlcourses);
        while ($row2 = mysqli_fetch_assoc($resultcourses)) {
            print "<option>" . $row2['CourseName'] . "</option>";
        }
    ?>
</select>
```

Figure 6.3 admin.php

The different home pages for administrators, students and lecturers are based around the same layout, with a navigation pane down the left-hand side and multiple frames that appear and disappear in the centre of the screen. Figure 6.3 above shows the add new student function built into a frame that appears on the screen. This frame will become hidden if a different option is selected.

6.3.4 Custom.css

```css
/*Course and Teacher rotating chevron glyphicon*/
#accordion .panel-heading { padding: 0;}
#accordion .panel-title > a {
    display: block;
    padding: 0.4em 0.6em;
    outline: none;
    text-decoration: none;
}
#accordion .panel-title > a.accordion-toggle::before, #accordion a[data-toggle="collapse"]::before  {
    content:"\e113";
    float: left;
    font-family: 'Glyphicons Halflings';
    margin-right :1em;
}
#accordion .panel-title > a.accordion-toggle.collapsed::before, #accordion a.collapsed[data-toggle="collapse"]::before  {
    content:"\e114";
}

/*Using a glyphicon as the bullet point*/
li {
    display: block;
}
#collapseOne li:before, #collapseTwo li:before , #collapse1 li:before, #collapse2 li:before, #collapse3 li:before, #collapse4 li:before, #collapse5 li:before{
    content: "\e080";
    font-family: 'Glyphicons Halflings';
    font-size: 12px;
    float: left;
    margin-top: 2px;
    margin-left: -17px;
    color: #CCCCCC;
}
```

Figure 6.3 custom.css

This controls how the site looks and controls how it is laid out. This css is used in conjunction with bootstrap css to make the site look clean and run smoothly.

6.3.5 Custom.js

```
/For hiding div panels on module, teacher and admin page
(document).ready(function (){
  $("#announcementBtn").click(function (){

      $("#announcementPanel").removeClass("invisible");

      $("#lecturePanel").addClass("invisible");
      $("#workShopPanel").addClass("invisible");
      $("#courseworkPanel").addClass("invisible");
      $("#studentPanel").addClass("invisible");
      $("#announcementPanel2").addClass("invisible");
      $("#lecturePanel2").addClass("invisible");
      $("#workShopPanel2").addClass("invisible");
      $("#courseworkPanel2").addClass("invisible");
      $("#studentPanel2").addClass("invisible");
  });
  $("#lectureBtn").click(function (){

      $("#lecturePanel").removeClass("invisible");

      $("#announcementPanel").addClass("invisible");
      $("#workShopPanel").addClass("invisible");
      $("#courseworkPanel").addClass("invisible");
      $("#studentPanel").addClass("invisible");
      $("#lecturePanel2").addClass("invisible");
      $("#announcementPanel2").addClass("invisible");
      $("#workShopPanel2").addClass("invisible");
      $("#courseworkPanel2").addClass("invisible");
      $("#studentPanel2").addClass("invisible");
  });
  $("#workshopBtn").click(function (){

      $("#workShopPanel").removeClass("invisible");
```

Figure 6.3 custom.js

This file controls the visibility of the frames on the admin, student and lecturer home pages. This means that the pages are not overloaded and makes sure the user does not have to scroll a long way down the page to find the information they require.

6.4 Issues and Problems Encountered

The main issued encountered was creating the javascript and getting it to function as expected. This took up the most amount of time to rectify as the solution was not obvious and required learning more javascript code. There were minor problems and speedbumps along the way such as the forms not submitting to the database or the reports not reading the correct information from the database or encountering an error, however, these were small issues that were quickly overcome.

6.5 Conclusion

In conclusion, the implementation of the system went pretty smoothly, and provided a suitable learning curve. The relative ease of the implementation meant that front-end parts of the system were quickly set up and ready for the back-end links to be put in place.

7 Testing

7.1 Testing Strategy

In software testing, there are two main approaches, static and dynamic. The static testing approach focuses on reviews, walkthroughs or inspections of the software, focusing mainly on the back-end code. This includes proofreading the code and reviewing the code. Dynamic testing focuses on the front-end of the system and is generally carried out whilst the code is running.

Software testing is generally carried out using a “box testing” method. There are three types of box testing: white, black and grey. White box testing focuses on testing the internal workings pf a system, requiring the tester to have an internal perspective of how the system is coded, programming skills and a working knowledge of how the system should behave. The tester creates test cases and chooses inputs to test the code and determine what the output is. Black box testing focuses on testing the system without the tester seeing any code or any knowledge of how the code works. The tester is provided with detailed use cases which details what inputs are to be entered and what the expected output is for each input. The tester uses these inputs and outputs to verify if the system works as expected. Grey box testing is a mixture of white and black box testing methods. Grey box testing involves having knowledge of the back-end code and how it works, using that knowledge to create tests then running those tests using the front-end. The tester is not required to have full access to the back-end code, but just enough to modify some features to fulfil the test cases.

To fully test the system, grey box testing is to be implemented as it will provide an outlet for testing both the back-end and the front-end. This will allow for the entire system to be fully tested and see that all functionality works as expected.

7.2 Testing Method

The grey box testing method can be split up into front-end tests and back-end tests. These may require different test cases or a different perspective on a single test case. This means that the testing can be done in two sets, with each set containing specific test cases.

Alongside the grey box testing method, user testing will be used to evaluate the accessibility and reliability of the system from a user’s point of view. To do this the system should be deployed to several volunteers to test and provide feedback. These volunteers would be recruited from family, friends and course mates, through the medium of email.

7.3 Testing

7.3.1 Back-end Testing

The back-end testing will encompass everything that allows the application to communicate with the back-end system. These tests will test the database and how it works.

Test Number	Test	Test Description	Expected Outcome	Actual Outcome
1	Front-end application connects to database	The front-end application successfully connects to the correct database	Connection is made and "Connected Successfully" is printed out on the screen	Connection was successfully made, and "Connected Successfully" was printed on the screen
2	Front-end application retrieves data	The front-end application successfully retrieves information from the database	The front-end displays the information retrieved from the database on a web page	Whilst the front-end did gather the correct information, it collected duplicates of the data. This was corrected by correcting the SQL statement

7.3.2 Front-end Testing

The Front-end testing will test any features that the web application has. Whilst this does not include testing the accessing and retrieval of data, it will include testing the methods used to instantiate the data access.

Test Number	Test	Test Description	Expected Outcome	Actual Outcome
1	Login	The user can login, and a session is set for that user.	The user is successfully logged in and their name appears at the top of the page to indicate	Whilst the user is successfully logged in, the session in not able to obtain the name and displays an error.

			whether the session has been successfully set or not.	
2	Login 2	The user can login, and a session is set for that user.	The user is successfully logged in and their name appears at the top of the page to indicate whether the session has been successfully set or not.	After making changes this test now passes.
3	Panel Visibility	The panels are hidden when not selected and visible when the relevant button pressed.	The panels that are not being used are hidden away so the user does not see them. Then when the user presses a button the visible panel is replaced by the relevant panel.	The panel disappears and the relevant panel displayed.
4	Registering for session	The user should be able to register for a specific session, and then see the attendance appear	The user successfully registers for the correct session and then is able to see that appear when they refresh the screen.	The user is able to register but it does not show up from the database.

8 Review

This chapter will evaluate the project picking through any problems encountered and discussing how things could be done differently if the project was undertaken again. The evaluation will start from the project conception phase and finish at the end of the implementation phase. As part of the evaluation the future, not just of this project but of many future projects, will be considered and discussed. Focussing on how the processes used in this project could be used again and developed. My own contributions to the project will be evaluated and any improvements will be discussed.

8.1 Project Evaluation

The project hit a slight snag compared to other projects from the off because there was no real client for the project, the only real client was myself. This meant that it was hard to establish what the projects specifications would be, and therefore, keep the project from becoming too large to manage. This also had an effect when it came to the analysis stage because, where other projects could speak to a client and get some real feedback on the idea and see what the system being built would actually do, this project had to be thought up and the specifications made up. Having said that, the specification became quite clear once research had been done. The research meant that the project now had an end point and had some boundaries to keep within, making it more manageable.

The project was mostly completed within the timescale, with some functionality not implemented in time. This was mostly due to lack of knowledge and time scale to learn the new skills. The project followed the software life cycle quite closely and made use of an agile methodology. This meant that the system would move from stage to stage fairly successfully, allowing for a structured approach to carrying out the project. The agile methodology allowed for some implementation to occur early on, allowing for the database to be built before any of the designs for the front end had been produced. This speeded up the process of designing and implementing the front end as it could be built around the database.

One major problem was the structure of the code base for the system. At the beginning of the project, the code was all in one folder and the PHP was mixed in with the HTML. This meant lots of repetitive code and lots of time lost correcting errors in multiple files. However, after being taught another way through a different module, a decision was made to try and separate the code out and make it easier to manage. This was both difficult to accomplish and caused a lot of issues to arise, mainly through the changes that had to be made to the code to accomplish the migration. The migration was only partly successful because of these issues and the code is still not totally separated.

8.2 Personal Evaluation

On a personal level this was a challenging project to undertake. There was lots of learning of new technologies and lots of research to do in order for the project to even begin. Adding that

to the lack of client meant that the project was a lot larger than originally anticipated. However, through thorough research and the continued help and support from my project supervisor, the project was kept on course and provided a large learning curve.

Despite every effort to produce a watertight system, there were several issues that were encountered on a personal level which slowed the progress of the project. These issues surrounded combining personal life with academic studies. This included working part-time, completing courseworks' for other subjects, dealing with illnesses and finding time to relax and de-stress. Whilst these were issues, they were overcome with varying degrees of success.

8.3 Future developments

If this project was to be done again, then there are some issues that would need to be ironed out to make the completion of the project happen more efficiently. For starters, the code would be written using MVC and separation of concerns from the outset, making the code easier to manage and easier to spot and deal with errors. If this project were to happen again, the finding of a real client for the system would be a huge help in establishing the critical functionality of the system and establishing the system end goals. This would make redoing the project more manageable and mean that the implementation of the system could start a lot sooner than it did.

Another development would be to use different frameworks to aid in the development and implementation of the system. This would allow for the developer to save time on writing code for different functions and allow for a much quicker output time on functionalities. This leads on to another major development for the future: adding new extended features to the system. These extended features could include a coursework uploading system, a system for lecturers to message students about attendance, coursework grading system, and more.

8.4 Conclusion

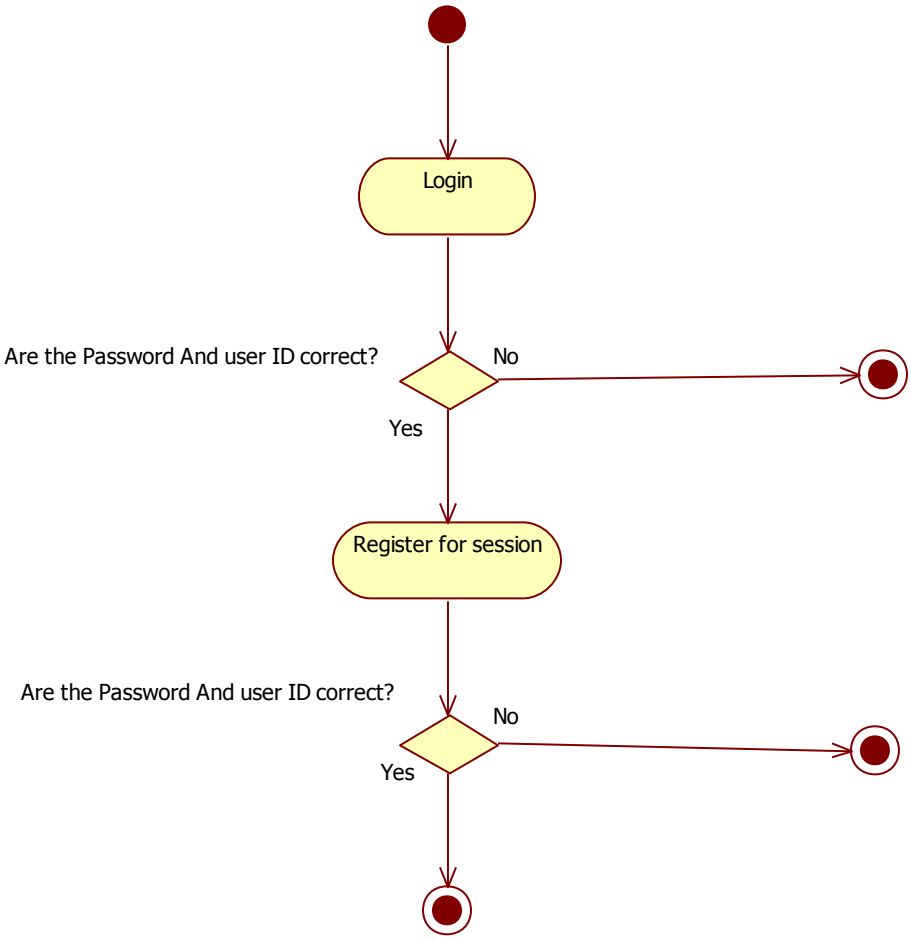
Overall the project was a success, whilst not all the functions originally specified were not completed on time the most important functions were completed. This meant that the system could be used for what it was intended.

The main thing taken away from this project was about how to effectively time manage and how to effectively plan tasks. This will allow for future projects to be undertaken in a more effective manner and more projects to be successfully completed and to a higher standard.

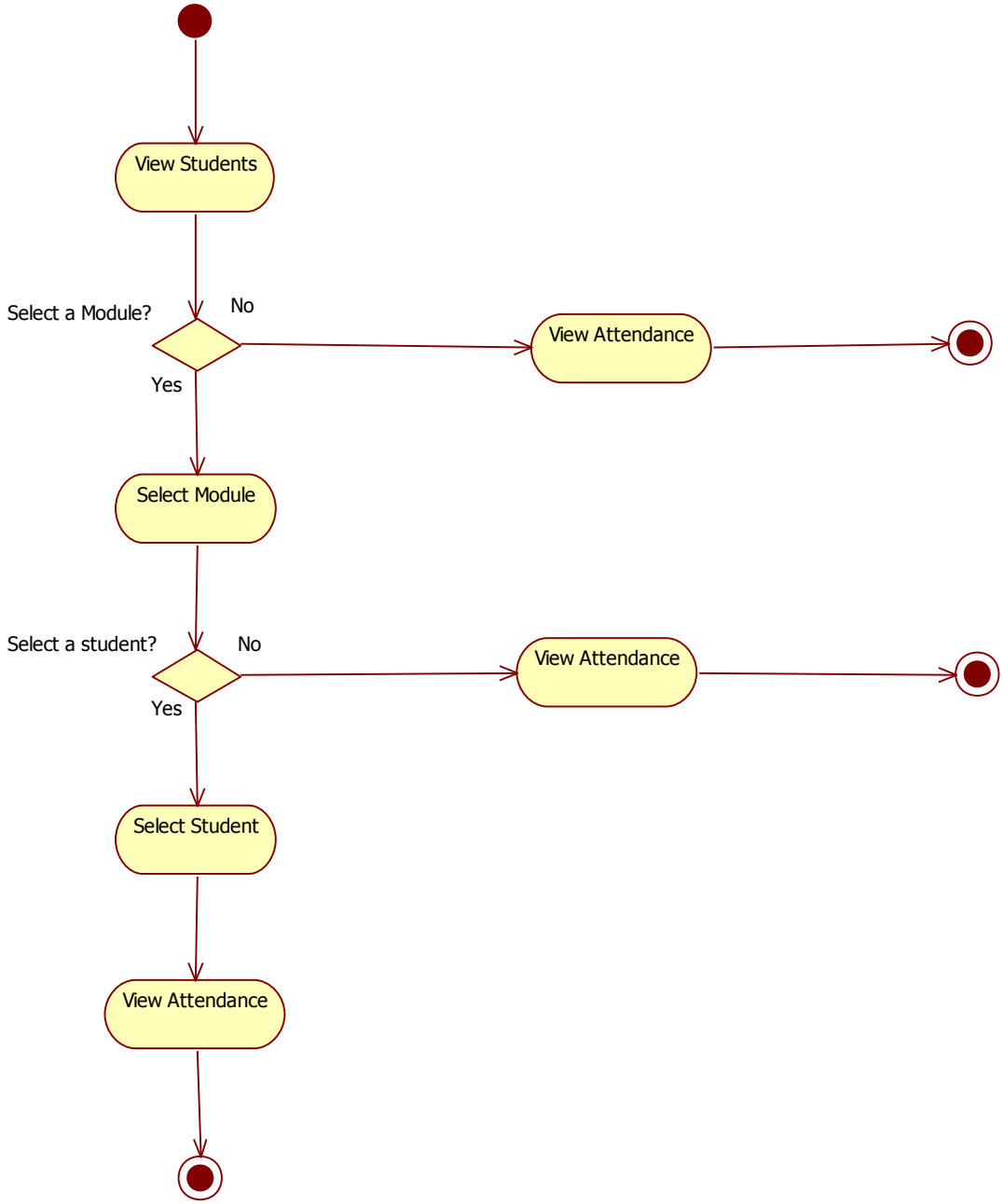
9 References

[1]	Thung, P. L., Ng, C. J., Thung, S. J., & Sulaiman, S. (2010). Improving a web application using design patterns: A case study. Information Technology (ITSim), 2010 International Symposium in. 1, p 1 - 6.
[2]	C. Alexander, S. Ishikawa, M. Silverstein, M. Jacobson, I. Fiksdahl-King, and S. Angel. A Pattern Language: Towns, Buildings, Construction. Oxford University Press, 1977.
[3]	Wiki.c2.com. (2014). *Smalltalk Language*. [online] Available at: http://wiki.c2.com/?SmalltalkLanguage [Accessed 5 Mar. 2017].
[4]	K. Beck and W. Cunningham. Using pattern languages for object-oriented programs. Technical Report CR-87-43, Tektronix, Inc., September 17, 1987. Presented at the OOPSLA'87 workshop on Specification and Design for Object-Oriented Programming.
[5]	Prajapati, H., & Dabhi, V. (2009). High Quality Web-Application Development on Java EE Platform. Advance Computing Conference, 2009. IACC 2009. IEEE International p 1664 – 1669.
[6]	A. Leff. J.T.Rayfield. Web_Application Development Using the Model/View/Controller Design Pattern
[7]	Christopher Alexander, Sara Ishikawa, MurraySilverstein, Max Jacobson, Ingrid Fiksdahl-King, and Shlomo Angel. A Pattern Language. Oxford University Press, NewYork, 1977
[8]	E. Gamma, R. Helm, R. Johnson, J. Vlissides. Elements of Reusable Object-Orientated Software

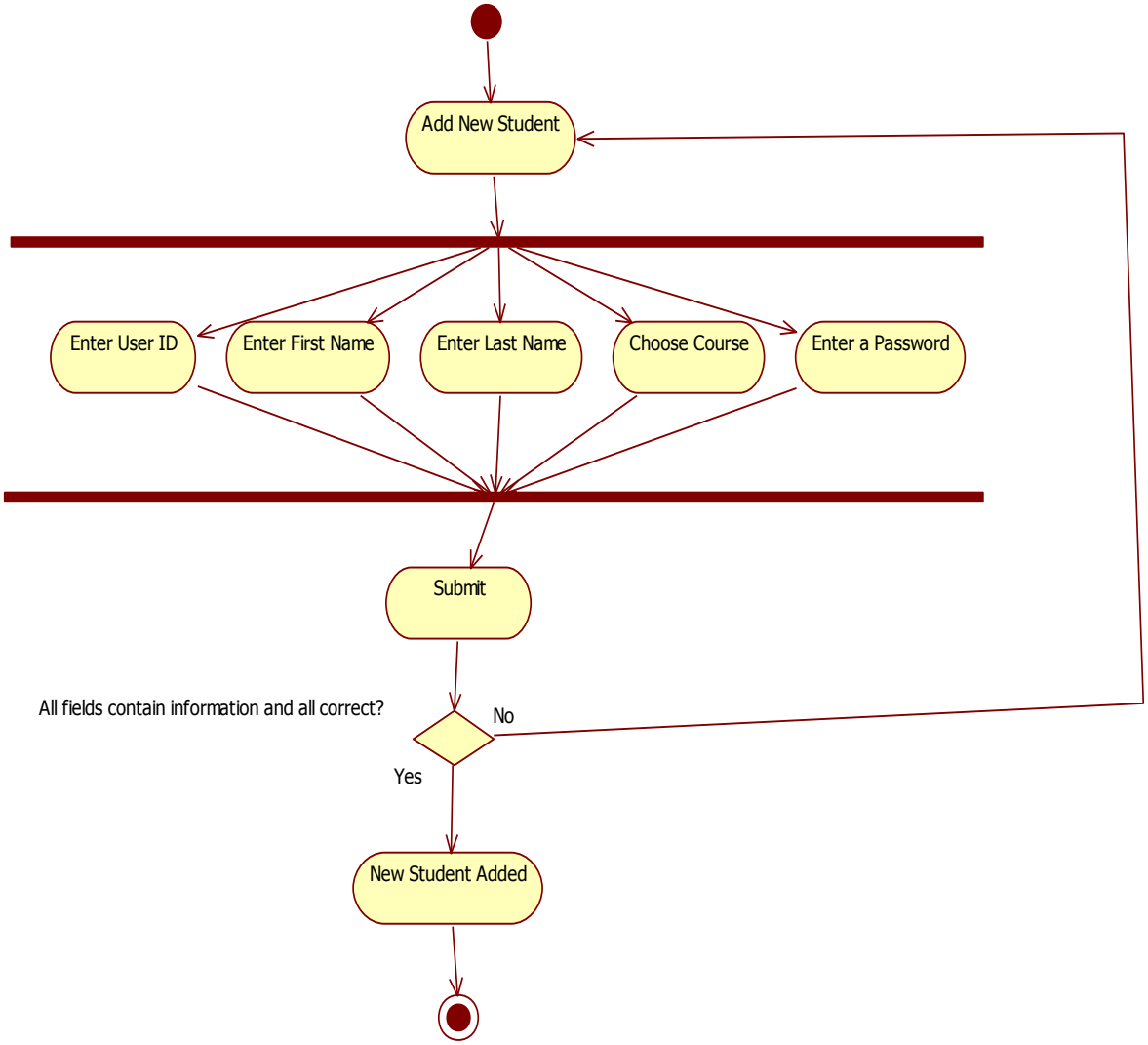
10 Appendix



Record Attendance



Lecturer view Attendance



Admin Add New Student

Table Name	Attribute Name	Description	Data Type	Range	Required	References	Notes
UserType	UserTypeID	ID of user Type	Int (11)		Y		
UserType	UserTypeID	ID of user Type	Int (11)		Y		
Users	UserID	The users ID	Varchar (8)		Y		Kxxxxxxx
	UserTypeID	ID of user Type	Int (11)		Y	UserType	
	Fname	The users forename	Varchar (15)				
	Lname	The users surename	Varchar (15)				
	Password	The users password	Varchar (20)				Encrypted by database
Courses	CourseID	The courses ID	Varcher (6)		Y		XXX999
	CourseName	The name of course of the system	Varchar (50)				
CourseModules	ModuleID	The modules ID	Varcher (6)		Y	Modules	
	CourseID	The id of course of the system	Varcher (6)		Y	Courses	
Modules	ModuleID	The modules ID	Varcher (6)		Y		XX9999

	ModuleName	The name of module of the system	Varchar (50)				
UserModules	UserID	The users ID	Varcher (6)		Y	Users	
	ModuleID	The id of module of the system	Varcher (8)		Y	Modules	
SessionType	SessionTypeID	ID of session Type	Int (5)				
	SessionType	The type of session of the system	Varcher (15)				
Sessions	SessionID	The sessions ID	Int (11)				
	SessionTypeID	ID of session Type	Int (5)		Y	SessionType	
	Date	Date of the Session	Date				
	Time	Time of the session	Varchar (5)				
	RoomNO	The no of room of the system	Varchar (8)				
Status	StatusID	The status ID	Int (11)		Y		
	StatusType	The type of status of the system	Varchar (1)	I, A			Constrained to values
	Reason	Reason for absense	Varchar (10000)				

Attendance	AttendanceID	The attendance ID	Int (5)		Y	
	AttendanceType	The type of attendance of the system	Varchar (1)	A, P		
	StatusID	The id of status of the system	Int (11)		Y	Status
SessionAttendance	SessionID	The sessions ID	Int (11)		Y	
	AttendanceID	The id of attendance of the system	Int (5)		Y	Attendance
UserSessions	UserID	The users ID	Varchar (8)		Y	Users
	SessionID	The id of session of the system	Int (11)		Y	Sessions
	AttendanceID	The id of attendance of the system	Int (5)		Y	Attendance
	ModuleID	The id of module of the system	Varcher (6)		Y	Modules